Nivaaran: Campus Grievance Redressal System

SUBMISSION DELIVERABLES & DEMONSTRATION LINKS

1. GitHub Repository: https://github.com/01-Aarambhika/nivaaran.git

2. Working Video Demo: https://drive.google.com/drive/folders/1jHVG6UMFhs6dGk2d55Yz0GA3Q7jfSy3L?usp=drive_link

1. Student Team & Service Ownership

Roll Number	Student Name	Service Ownership Scope	Core Technical Responsibilities
20251651006	Abhishek	Complaints Service (Port 8082) Resolution Service (Port 8083)	Resource modeling, grievance state transitions, staff allocation logic, escalation path design.
20251651001	Aarambhika	Accounts Service (Port 8080) Directory Service (Port 8081)	Identity & RBAC contract design, JWT token verification specs, category SLA matrices, venue master data.
20251651078	Ritik	Complaints Service (Port 8082) Notifications Service (Port 8084)	Core REST backend implementation, Node test suite (11/11 pass), HTTP headers, ETag concurrency, rate limiting, cURL transcripts.
Full Team	Ritik, Abhishek, Aarambhika	Analytics & SLA Service (Port 8085) React Frontend SPA (Port 5173)	System-wide audit event ingestion, SLA compliance KPI tracking, React client web interface integration.

2. Executive Summary & Problem Statement

At the IIIT Vadodara campus, students facing day-to-day residential and academic infrastructure issues (such as water supply disruptions, broken electrical fixtures in hostel rooms, mess food hygiene complaints, or campus Wi-Fi outages) historically had to report grievances through informal, manual channels: verbal intimations, paper slips, or ad-hoc WhatsApp messages.

This traditional workflow caused severe operational drawbacks:

- **Lack of Accountability:** Complaints were routinely lost or forgotten with no formal log of submission time.
- **No Defined Ownership:** Tickets bounced between departments (e.g. warden vs. maintenance) without designated staff responsibility.
- **Absence of Deadlines:** Small grievances remained unresolved for weeks due to missing Service Level Agreements (SLAs).
- **Zero Student Visibility:** Complainants had no means to track progress without physically chasing institute staff.

Nivaaran ("remedy" or "redress") resolves these challenges by providing an institutional grievance redressal pipeline where complaints are formally ingested, validated against campus directories, tracked through deterministic state machines, and automatically escalated if resolution deadlines are breached

3. Implementation Scope vs. System Architecture Scope

To satisfy both enterprise architectural standards and the practical scope requirements of the coursework:

- **Full System Architecture (6 Microservices):** The entire platform is formally designed across six autonomous microservices with isolated database boundaries, OpenAPI contracts, and relational ER schemas (documented in docs/a2/design.pdf).
- **Core Runnable Service (Complaints Service):** In strict accordance with assignment instructions, the Complaints Service (services/complaints/) is the single core runnable backend microservice fully coded, running on Node.js/Express (Port 8082), and validated with 11 automated unit and integration tests.

4. Assignment 01: Project Setup, Team Roles & Git Collaboration

Assignment 01 established the institutional problem brief, noun/verb system models, user role boundaries, and team collaboration workflows:

- **Student Role:** Signs up using roll number; files grievances, tracks status, adds comments, rates resolution, and escalates overdue tickets.
- **Staff Role:** Signs in with Staff ID; resolves grievances within their assigned department (Warden, Mess In-Charge, Maintenance Staff).
- **Admin Role:** Creates staff accounts, assigns tickets, escalates overdue cases, and monitors institute-wide SLA compliance telemetry.
- **Git Collaboration Workflow:** A strict Git Flow workflow was implemented. Direct pushes to main are forbidden. All work proceeds on feature branches (feature/complaints-api, feature/http-headers), requiring peer code reviews and CI test validation before merging.

5. Assignment 02: 6-Microservices Architecture & Database Schemas

Assignment 02 decomposed the system capabilities into 6 autonomous services adhering to the Database-per-Service architectural pattern:

#	Service Name	Port	Owned Database	Core Domain Responsibility
1	**Accounts**	8080	nivaaran_accounts	User identity, roll numbers, credentials, bcrypt password hashes, JWT role tokens.
2	**Directory**	8081	nivaaran_directory	6 Campus grievance categories, SLA hour limits, campus zone and venue master data.
3	**Complaints**	8082	nivaaran_complaints	Core Implemented Service: Grievance lifecycle, state transitions, comments, feedback.
4	**Resolution**	8083	nivaaran_resolution	Staff allocations, duty rosters, department queue workloads, shift assignments.
5	**Notifications**	8084	nivaaran_notifications	Outbound SMS logs, push notification queue, carrier dispatch delivery receipts.
6	**Analytics & SLA**	8085	nivaaran_analytics	Central immutable audit trail, daily SLA compliance KPIs, escalation breach reports.

Database Isolation & Soft-References: Cross-database foreign keys and synchronous joins across microservice boundaries are strictly prohibited. Services associate records through immutable logical string keys (reporter_id, category_id). This prevents cascading failure propagation and allows independent schema evolution. The complete schema is modeled in docs/a2/schema.sql, docs/a2/schema.png, and docs/a2/services.png.

6. Assignment 03: SOAP & External XML Integration

Assignment 03 modeled inter-system enterprise integration with legacy external providers. Nivaaran integrates with a National SMS Telecom Gateway to dispatch emergency mobile SMS alerts to hostel wardens and students:

- **WSDL Contract (docs/a3/partner.wsdl):** Defines the SOAP 1.1 RPC service containing the sendSMS operation, binding over HTTP with typed schemas.
- **SOAP Request (docs/a3/soap-request.xml):** Carries SOAP Envelope and Body containing recipientPhone, senderId, and messageContent.
- **SOAP Response (docs/a3/soap-response.xml):** Delivers transaction reference SMS-2026-99214 and carrier receipt timestamp.
- **SOAP Fault (docs/a3/soap-fault.xml):** Standardized XML fault handling with <faultcode>Client.InvalidPhoneFormat</faultcode> when recipient numbers fail ITU E.164 formatting.
- **Architectural Audit (WSDL vs. OpenAPI):** The WSDL file required 94 lines of XML ceremony for a single operation, whereas OpenAPI concisely describes richer validation, query parameters, and diverse HTTP status codes in lightweight YAML.

7. Assignment 04: Rebuilding Service in REST & OpenAPI 3.0 Contract

Assignment 04 converted the grievance intake interface into a proper RESTful architecture. The complete OpenAPI contract is formalized in services/complaints/openapi.yaml and docs/a4/openapi.yaml.

Method	URL Endpoint	Operational Purpose	Success Code	Failure Codes
POST	/complaints	File a new grievance (supports Idempotency-Key)	201 Created (200 on retry)	400, 401, 422, 503
GET	/complaints/{id}	Read single complaint (supports If-None-Match)	200 OK / 304 Not Mod.	401, 404
GET	/complaints	Filter list by category, status, and pagination	200 OK	400, 401
PATCH	/complaints/{id}	Partial status update (requires If-Match)	200 OK	401, 404, 412, 422
POST	/complaints/{id}/escalate	Escalate overdue grievance (state action)	200 OK	401, 404, 409, 422
POST	/complaints/{id}/comments	Append discussion note to thread	201 Created	400, 401, 404, 422
DELETE	/complaints/{id}	Withdraw submitted draft complaint	204 No Content	401, 404, 409
OPTIONS	/complaints	CORS preflight & allowed methods discovery	204 No Content	—

Justification of Hard REST Design Choice: Escalation

In legacy RPC systems, escalating a complaint was modeled as a procedure call: escalateComplaint(id, reason). In REST, URLs must represent durable resources (nouns). We evaluated three modeling designs:

- **Collection (POST /escalations):** Rejected because escalation has no independent lifecycle outside of a complaint.
- **Generic Patch (PATCH /complaints/{id} with {"isEscalated": true}):** Rejected because escalation is a state-changing business transaction that enforces SLA breach prerequisites, records immutable audit trails, and must deterministically reject duplicates with 409 Conflict.

• **State-Changing Sub-Resource (POST /complaints/{id}/escalate):** Selected. Preserves clean URI hierarchy, treats the escalation action as an operation on the grievance, and allows proper HTTP status semantics (200 OK on initial escalation, 409 Conflict if already escalated).

Outbound Resilience, Circuit Breaker & Fallback (accountsClient.js)

When Complaints calls Accounts (GET /users/{id}) to verify student enrollment, the call is protected by:

1. Timeout: Strict 2.0-second abort controller.

2. Exponential Backoff with Jitter: Up to 3 attempts with randomized delay for transient network glitches (4xx client errors are never retried).

3. Circuit Breaker (src/breaker.js): Trips open after 5 consecutive failures, fast-failing for 30 seconds.

4. Graceful Degradation Fallback: If Accounts is offline, Complaints accepts the ticket with user_validation_pending: true, returning 201 Created. Rationale: A student reporting an active burst pipe or electrical fire hazard must never be blocked because an auxiliary service has a temporary restart.

8. Assignment 05: HTTP Methods, Headers, Caching & Idempotency

Assignment 05 implemented complete HTTP specification semantics to guarantee safe network retries and high performance:

1. The Safe-Retry Plan (Idempotency-Key)

On mobile campus networks, clients often lose connection after the server commits a record but before the HTTP response arrives. The student device automatically retries, risking duplicate tickets and redundant staff dispatch:

- The client generates a unique UUID header: Idempotency-Key: 8f4a-912b-47e1.
- On first submission, Complaints creates the ticket, caches the response, and returns 201 Created with a Location: /complaints/cmp_1001 header.
- On repeat submission with the same key, the service detects the cached key and short-circuits execution, returning the original representation with 200 OK without duplicating database records or triggering redundant alerts.

2. Optimistic Concurrency Locking (ETag & If-Match)

Prevents the Lost Update Problem when multiple staff members view and update the same complaint concurrently:

- Every read returns a strong cryptographic hash header: ETag: "1-a8f9c1".
- Mutating requests (PATCH /complaints/{id}) must include If-Match: "1-a8f9c1".
- If the complaint was modified in the interim, the incoming ETag mismatches current state, and Complaints immediately refuses the update with 412 Precondition Failed, protecting data integrity.

3. Conditional Bandwidth Optimization (If-None-Match)

When mobile clients poll complaints, they send If-None-Match: "1-a8f9c1". If the record has not changed, the server responds with 304 Not Modified with an empty body, eliminating bandwidth waste.

9. Assignment 06: Honest Responses, Status Codes & RFC 7807 Errors

Assignment 06 eliminated generic responses by implementing strict HTTP status codes and RFC 7807 Problem Details:

Problem Details Format (application/problem+json)

```
{ "type": "/errors/precondition-failed", "title": "Precondition Failed", "status": 412, "detail": "Resource modified concurrently. Current ETag is \"2-d7b3e4\"; requested was \"1-a8f9c1\". Refresh before editing." }
```

Field-Level Validation (422 Unprocessable Entity)

When client input contains domain validation errors, Complaints returns all errors simultaneously in a structured list:

```
{ "type": "/errors/validation-failed", "title": "Validation Failed", "status": 422, "detail": "Request body failed 2 business validation constraints", "errors": [ { "field": "title", "reason": "Title must be at least 10 characters" }, { "field": "description", "reason": "Description must be at least 20 characters" } ] }
```

Content Negotiation (Accept Header)

- Requests with Accept: application/json receive JSON representations.
- Requests with Accept: application/xml receive well-formed XML payloads with matching schemas.
- Requests requesting unsupported MIME types (e.g., text/csv) are rejected with 406 Not Acceptable.

10. Automated Testing Suite Verification (11/11 Tests Passing)

The Complaints microservice contains a comprehensive automated test suite (services/complaints/tests/complaints.test.js) executed via the native Node.js Test Runner (node --test). All 11 tests pass with 100% success rate:

#	Test Case Description	Targeted Architectural Requirement	Execution Status
1	POST /complaints creates complaint with 201, Location, and ETag	Assignment 04 & 05: Resource Creation & Headers	**PASSED (538ms)**
2	Idempotent repeat with same Idempotency-Key returns original 200	Assignment 05: Duplicate-Proof Safe Retries	**PASSED (408ms)**
3	Missing Authorization header returns 401 with WWW-Authenticate	Assignment 06: Honest Authentication Errors	**PASSED (7ms)**
4	Field-level 422 reports two bad fields simultaneously	Assignment 06: RFC 7807 Multi-Field Validation	**PASSED (8ms)**
5	Conditional GET with matching If-None-Match returns 304 Not Modified	Assignment 05: HTTP Bandwidth Caching	**PASSED (419ms)**
6	Conditional PATCH with mismatched If-Match returns 412 Precondition Failed	Assignment 05: Optimistic Concurrency Control	**PASSED (384ms)**
7	Conditional PATCH with matching If-Match succeeds and updates ETag	Assignment 05: Atomic State Evolution	**PASSED (403ms)**
8	DELETE /complaints/:id returns 204 No Content	Assignment 05: Idempotent Resource Removal	**PASSED (14ms)**
9	OPTIONS /complaints returns 204 with Allow and CORS headers	Assignment 05: Method Discovery & CORS Preflight	**PASSED (3ms)**
10	Content Negotiation: XML succeeds when asked; unsupported returns 406	Assignment 06: MIME Type Negotiation	**PASSED (11ms)**

11	Escalate already escalated complaint returns 409 Conflict	Assignment 04 & 06: Lifecycle State Conflict	**PASSED (12ms)**

Test Execution Architecture & Methodology

The test suite executes against an in-process Express application instance using Node's native HTTP assertion harness. Tests are self-contained and run without external database or network dependencies:

- **Atomic State Isolation:** In-memory repository stores (src/store.js) are initialized before each test block, ensuring zero side-effects across test runs.
- **Deterministic Mocking:** Cross-service HTTP calls to the Accounts Service are intercepted through dependency-injected mock handlers (mockHandler option in accountsClient.js) to test both 200 OK verification and 503 fallback scenarios deterministically.
- **Zero Third-Party Test Bloat:** Built using Node.js 20+ built-in test runner (node --test) and strict assertions (node:assert), eliminating external testing framework vulnerabilities.

Automated Verification Command: Execute "npm test" inside services/complaints/. Total runtime is ~2.2 seconds across all 11 test suites with 0 failures, 0 skipped, and 0 warnings.

11. Local Setup & Step-by-Step Reproduction Guide

The repository can be cloned and executed locally on any machine with Node.js 20+ installed:

1. Clone Repository & Install Dependencies

```
git clone https://github.com/ritik-78/Nivaaran-Grievance-Redressal.git cd Nivaaran/services/complaints npm install
```

2. Run Automated Test Suite

```
npm test # Result: 11 tests pass cleanly (node:test)
```

3. Start Complaints Microservice

```
npm start # Server listens on port 8082: http://localhost:8082
```

4. End-to-End cURL Execution Verification Scenarios

Scenario A: File a Grievance with Safe Idempotency (Status 201 Created)

```
curl -i -X POST http://localhost:8082/complaints \  -H "Content-Type: application/json" \  -H "Authorization: Bearer mock-token-20251651078" \  -H "Idempotency-Key: uuid-test-1001" \  -d '{"userId":"20251651078","categoryId":"hostel","locationId":"hostel_block_a",  "title":"Water leakage in room 302","description":"Bathroom tap broken continuously dripping.",  "priority":"urgent"}'
```

Scenario B: Idempotent Retry (Returns Cached 200 OK without Duplicating)

```
curl -i -X POST http://localhost:8082/complaints \  -H "Content-Type: application/json" \  -H "Authorization: Bearer mock-token-20251651078" \  -H "Idempotency-Key: uuid-test-1001" \  -d '{"userId":"20251651078","categoryId":"hostel","locationId":"hostel_block_a",  "title":"Water leakage in room 302","description":"Bathroom tap broken continuously dripping.",  "priority":"urgent"}'
```

Scenario C: Conditional Update with Optimistic Locking (ETag Concurrency)

```
curl -i -X PATCH http://localhost:8082/complaints/cmp_1001 \  -H "Content-Type: application/json" \  -H "Authorization: Bearer mock-token-20251651078" \  -H "If-Match: \"1-a8f9c1\"" \  -d '{"status":"in_progress","remarks":"Technician assigned and dispatched."}'
```

12. Submission Summary & Deliverables Checklist

The Nivaaran project demonstrates a complete, mathematically rigorous realization of Service-Oriented Architecture principles for campus grievance redressal. All questions and technical requirements across Assignments 01 to 06 have been thoroughly designed, implemented, tested, and documented.

- **GitHub Repository:** Clean codebase, no node_modules committed, modular layout, complete README.md and ReadMe.docx.
- **ReadMe Documents :** Detailed documentation of Assignments 01 to 06, GitHub link, Google Drive demo link, reproduction guide.
- **Google Drive Video:** Step-by-step working video demonstration of all REST endpoints, automated test run, and React client integration shared with public view access.